

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_org = pd.read_excel("/content/drive/MyDrive/capstone/datasets/0_original.xlsx")
df_org.head()
```

	data_source	full_name	age	age_group	family_members	position_in_siblings	gender	maritual_status	duration_of_separataion	profession_group
0	https://www.dhakatribune.com/bangladesh/nation...	Parvez	NaN	Teen	NaN	NaN	Male	NaN	NaN	Worke
1	https://www.dhakatribune.com/bangladesh/nation...	Alhaj Uddin	20.0	Teen	NaN	NaN	Male	relationship-rejected	NaN	NaN
2	https://www.dhakatribune.com/bangladesh/nation...	Priti Akter	14.0	Teen	NaN	NaN	Female	unmarried	NaN	Studen
3	https://www.dhakatribune.com/bangladesh/nation...	Acchia Akhter	NaN	NaN	NaN	NaN	Female	relationship-rejected	NaN	Studen
4	https://www.dhakatribune.com/bangladesh/dhaka/...	Dilshad Nahar	18.0	Teen	NaN	NaN	Female	relationship	NaN	Studen

5 rows × 36 columns

```
df_clr = pd.read_excel("/content/drive/MyDrive/capstone/datasets/1_cleaned.xlsx")
df_clr.head()
```

	age	age_group	gender	marital_status	profession_group	profession_description	workplace	religion	financial_condition	hometown	...	feels_like	temp_min	temp_max
0	NaN	teen	male	NaN	worker	he was an plumber	rural	muslim	indebted	brahmanbaria	...	302.86	299.15	299.15
1	20.0	teen	male	rejected	NaN	NaN	NaN	muslim	NaN	sylhet	...	299.17	295.57	295.57
2	14.0	teen	female	single	student	student of class nine	urban	muslim	NaN	manikganj	...	304.33	301.15	301.15
3	NaN	NaN	female	rejected	student	student of Bangla Department	urban	muslim	NaN	bogura	...	303.67	299.34	299.34
4	18.0	teen	female	relationship	student	studied fashion design at a private university	urban	muslim	NaN	dhaka	...	304.49	300.15	300.15

5 rows × 28 columns

df_clr.columns

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
      'profession_description', 'workplace', 'religion',
      'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
      'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
      'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
      'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
      'weather_description'],
      dtype='object')
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Create a DataFrame for the missing data summary
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage']]
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage']]
    group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage']]

    # Sort each group by missing percentage in descending order (most missing to least missing)
    group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

    # Rename columns for clarity
    group_1.columns = ['Column', 'Missing %']
    group_2.columns = ['Column', 'Missing %']
    group_3.columns = ['Column', 'Missing %']
    group_4.columns = ['Column', 'Missing %']
```

```
# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))
```

```
missing_data_report(df_clr)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %]
Index: []
```

```
Group 2: Columns with >40% to ≤70% missing data
```

Column	Missing %
temp_min	53.061224
feels_like	53.061224
clouds_sky	53.061224
wind_deg	53.061224
air_humidity	53.061224
air_pressure	53.061224
temp_max	53.061224
latitude	52.999382
wind_speed	52.999382
weather_main	52.999382
longitude	52.999382
weather_description	52.999382
unix_time	52.380952
profession_description	49.412492
financial_condition	45.516388

```
Group 3: Columns with >20% to ≤40% missing data
```

Column	Missing %
workplace	37.909709
marital_status	23.933210

```
Group 4: Columns with ≤20% missing data
```

Column	Missing %
profession_group	18.923933
reason	16.821274
reason_description	16.635745
age	12.739641
time	12.059369
age_group	7.421150
method	2.906617
religion	2.411874
suicide_date	2.040816
hometown	1.669759
gender	0.432900

```
categorical_columns = [
    'age_group', 'gender', 'religion', 'marital_status', 'profession_group', 'workplace',
    'financial_condition', 'reason', 'time', 'method', 'hometown', 'weather_main', 'weather_description',
]
```

```
descriptive_columns = ['profession_description', 'reason_description']

numerical_columns = [
    'age', 'latitude', 'longitude', 'unix_time', 'feels_like', 'temp_min',
    'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky'
]

weather_columns = [
    'unix_time', 'feels_like', 'temp_min', 'temp_max', 'air_pressure',
    'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
    'weather_main', 'weather_description', 'latitude', 'longitude'
]

# only date format in ths database is "suicide_date"
```

```
def print_unique_values(df, column_name):
    if column_name in df.columns:
        unique_values = df[column_name].unique()
        print(f"Unique values in column '{column_name}':")
        print(unique_values)
    else:
        print(f"Column '{column_name}' not found in the DataFrame.")

print_unique_values(df_clr, 'weather_description')
```

```
Unique values in column 'weather_description':
['haze' 'scattered clouds' 'drizzle' 'moderate rain' 'overcast clouds'
'broken clouds' 'clear sky' 'light rain' 'mist' 'few clouds'
'heavy intensity rain' 'thunderstorm with rain' 'thunderstorm'
'thunderstorm with light rain' nan 'cloudy']
```

```
!pip install geopy
```

```
Requirement already satisfied: geopy in /usr/local/lib/python3.11/dist-packages (2.4.1)
Requirement already satisfied: geographiclib<3,>=1.52 in /usr/local/lib/python3.11/dist-packages (from geopy) (2.0)
```

```
import pandas as pd
import time
import json
from geopy.geocoders import Nominatim
from tqdm.notebook import tqdm # use tqdm.notebook for Colab/Jupyter
import os

# Step 1: Initialize geolocator
geolocator = Nominatim(user_agent="myGeolocator", timeout=10)

cache_file = "/content/drive/MyDrive/capstone/hometown_coords.json"

if os.path.exists(cache_file) and os.path.getsize(cache_file) > 0:
    with open(cache_file, "r") as f:
        hometown_coords = json.load(f)
else:
    hometown_coords = {}
```

```
# Step 3: Get unique hometowns from df_org
unique_hometowns = df_org['hometown'].dropna().str.strip().str.lower().unique()

# Step 4: Geocode with progress bar and update cache
for hometown in tqdm(unique_hometowns, desc="Geocoding hometowns"):
    if hometown not in hometown_coords or hometown_coords[hometown] == [None, None]:
        try:
            location = geolocator.geocode(hometown + ', Bangladesh')
            if location:
                hometown_coords[hometown] = [location.latitude, location.longitude]
            else:
                hometown_coords[hometown] = [None, None]
        except Exception as e:
            print(f"Error geocoding '{hometown}': {e}")
            hometown_coords[hometown] = [None, None]
        time.sleep(1) # Nominatim rate limit

# Step 5: Save updated cache
with open(cache_file, "w") as f:
    json.dump(hometown_coords, f)

# Step 6: Map latitude and longitude from cache to df_clr
def get_lat(row):
    ht = str(row['hometown']).strip().lower()
    return hometown_coords.get(ht, [None, None])[0]

def get_lon(row):
    ht = str(row['hometown']).strip().lower()
    return hometown_coords.get(ht, [None, None])[1]

df_clr['latitude'] = df_clr.apply(get_lat, axis=1)
df_clr['longitude'] = df_clr.apply(get_lon, axis=1)
```

Geocoding hometowns: 100%

593/593 [11:54<00:00, 1.25s/it]

```
# Show the tail of the latitude and longitude columns
display(df_clr[['latitude', 'longitude']].tail(5))
```

	latitude	longitude
1612	NaN	NaN
1613	NaN	NaN
1614	NaN	NaN
1615	23.898795	89.129554
1616	22.655478	89.794181

```
missing_data_report(df_clr)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %]
```

Index: []

Group 2: Columns with >40% to ≤70% missing data

Column	Missing %
temp_min	53.061224
feels_like	53.061224
air_humidity	53.061224
air_pressure	53.061224
temp_max	53.061224
clouds_sky	53.061224
wind_deg	53.061224
weather_main	52.999382
wind_speed	52.999382
weather_description	52.999382
unix_time	52.380952
profession_description	49.412492
financial_condition	45.516388

Group 3: Columns with >20% to ≤40% missing data

Column	Missing %
workplace	37.909709
marital_status	23.933210

Group 4: Columns with ≤20% missing data

Column	Missing %
profession_group	18.923933
reason	16.821274
reason_description	16.635745
age	12.739641
time	12.059369
age_group	7.421150
latitude	6.493506
longitude	6.493506
method	2.906617
religion	2.411874
suicide_date	2.040816
hometown	1.669759
gender	0.432900

prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

Make the latest DataFrame downloadable

from google.colab import files

Specify the filename for the Excel file

filename = '2_lat_lon.xlsx'

Save the DataFrame to an Excel file

df_clr.to_excel(filename, index=False)

Download the file

files.download(filename)

